



AXI DMA Controller: Relatório Final de Projeto Fase II

Vinícius Silva Oliveira

29 de março de 2026

Sumário

1	Introdução e Especificação	1
1.1	Resumo Executivo	1
1.2	Arquitetura do Sistema	1
1.3	Tabela de Interfaces	2
1.4	Conformidade com AXI4 e AXI-Lite	4
1.5	Mapa de Registradores	5
2	Implementação RTL	6
2.1	Estratégia de Design	6
2.2	Hierarquia do Projeto	6
2.3	Máquina de Estados (FSM)	7
2.4	Motores AXI	8
2.5	Pipeline e Desacoplamento de Dados	8
2.6	Considerações de Projeto para ASIC	9
2.7	Trechos Relevantes de Código	10
3	Estratégia de Verificação e Resultados	11
3.1	Ambiente UVM	11
3.2	Golden Model e Scoreboard	12
3.3	Verificação Baseada em Assertions (SVA)	12
3.4	Análise de Cobertura	13
3.4.1	Code Coverage	13
3.4.2	Functional Coverage	13
3.5	Resultados de Simulação RTL	14
4	Síntese Lógica e Resultados PPA	15
4.1	Fluxo de Síntese	15
4.2	Restrições de Projeto	16
4.3	Validação Pós-Síntese por Gate-Level Simulation	16
4.3.1	Análise do Problema	17
4.4	Resultados de Timing	17
4.5	Discussão de Área	18
4.6	Discussão de Potência	19
5	Conclusão e Lições Aprendidas	19

Lista de Figuras

1	Diagrama de blocos do AXI DMA	1
2	Máquina de estados do controlador DMA.	7
3	Waveform representativa de uma transferência DMA em burst, evidenciando a coordenação entre os canais AXI de leitura e escrita.	15

Lista de Tabelas

1	Interfaces do módulo dma_axi_top.	3
2	Mapa de registradores do controlador DMA.	5

1 Introdução e Especificação

1.1 Resumo Executivo

Este projeto implementa um controlador DMA (*Direct Memory Access*) do tipo *Memory-to-Memory* (MM2MM) com interface baseada em AXI, descrito em SystemVerilog. O bloco foi desenvolvido com foco em aprendizado, exploração arquitetural e prática de fluxo de front-end para ASICs e FPGAs, abrangendo especificação, implementação RTL, verificação, síntese lógica e simulação *gate-level*.

O controlador tem como finalidade realizar transferências em *burst* entre duas regiões de memória mapeadas, reduzindo a intervenção do processador na movimentação de dados. A configuração do bloco é feita por meio de uma interface AXI4-Lite de registradores de controle e status, enquanto a transferência propriamente dita utiliza duas interfaces mestre AXI4 separadas: uma para leitura da memória de origem e outra para escrita na memória de destino.

Do ponto de vista funcional, o projeto suporta transferências MM2MM com descritor único, apenas uma transação pendente por vez, *burst* incrementais (INCR) e comprimento de *burst* configurável. Por outro lado, recursos mais avançados de DMA, como *scatter-gather*, múltiplos canais, AXI-Stream, execução fora de ordem e transferências desalinhadas, não fazem parte do escopo desta implementação.

1.2 Arquitetura do Sistema

A arquitetura do sistema é composta por seis blocos principais: o módulo de topo (`dma_axi_top`), a máquina de estados de controle (`dma_fsm`), o motor de leitura AXI (`dma_axi_master_rd`), o motor de escrita AXI (`dma_axi_master_wr`), a FIFO de desacoplamento (`dma_fifo`) e o bloco de registradores de controle e status (`dma_csr`). Uma visão geral da arquitetura pode ser observada na Figura 1.

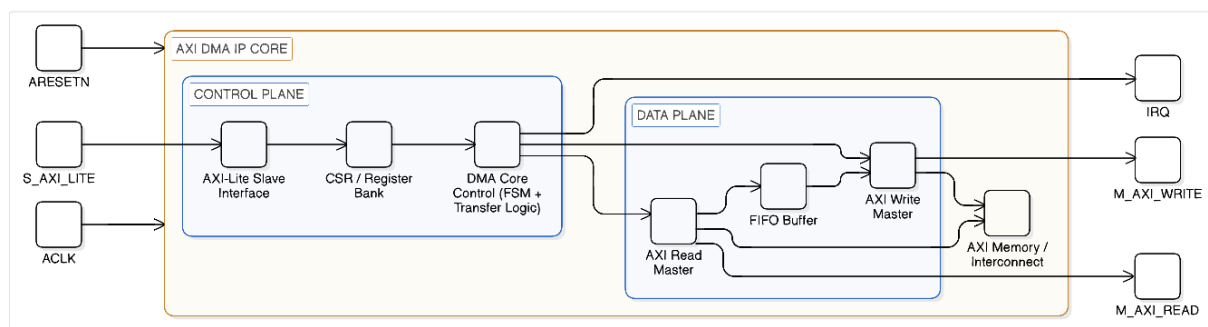


Figura 1: Diagrama de blocos do AXI DMA

A operação do controlador segue o fluxo abaixo:

-
1. o escravo configura os registradores de origem, destino, tamanho da transferência e tamanho máximo de burst via AXI4-Lite;
 2. o registrador de controle gera um pulso de início (`start_pulse`);
 3. a FSM principal calcula o burst corrente e emite um comando para o motor de leitura;
 4. o módulo de leitura realiza a transação no canal AR/R da interface AXI4 mestre de leitura;
 5. os dados recebidos são armazenados em uma FIFO interna, desacoplando temporariamente os canais de leitura e escrita;
 6. quando há dados suficientes disponíveis, a FSM emite um comando para o motor de escrita;
 7. o motor de escrita consome os dados da FIFO e realiza a transação nos canais AW/W/B da interface AXI4 mestre de escrita;
 8. ao final da transferência, a FSM sinaliza conclusão ou erro, atualizando os registradores de status e interrupção.

A solução adota uma arquitetura simples e sequencial, com apenas um burst ativo por vez em cada módulo, sem suporte a múltiplas transações sobrepostas ou reordenação. Essa escolha reduz complexidade e facilita verificação, porém limita throughput máximo, pois não há sobreposição de bursts nem múltiplas transações outstanding, o que impede o aproveitamento completo do paralelismo do protocolo AXI.

1.3 Tabela de Interfaces

O módulo de topo `dma_axi_top` expõe uma interface síncrona com um único clock (ACLK) e um reset ativo em nível baixo (ARESETn). As interfaces externas podem ser agrupadas em três conjuntos: AXI4-Lite escravo para configuração, AXI4 mestre de leitura e AXI4 mestre de escrita, além de uma saída de interrupção (IRQ).

Tabela 1: Interfaces do módulo dma_axi_top.

Sinal	Direção	Largura	Descrição
Clock e Reset			
ACLK	In	1	Clock principal do sistema
ARESETn	In	1	Reset global ativo em nível baixo
AXI4-Lite Slave (Configuração)			
S_AXI_AWADDR	In	32	Endereço de escrita
S_AXI_AWVALID	In	1	Validação do endereço de escrita
S_AXI_AWREADY	Out	1	Pronto para receber endereço
S_AXI_WDATA	In	32	Dados de escrita
S_AXI_WSTRB	In	4	Máscara de byte
S_AXI_WVALID	In	1	Validação dos dados
S_AXI_WREADY	Out	1	Pronto para receber dados
S_AXI_BRESP	Out	2	Resposta de escrita (fixo OKAY)
S_AXI_BVALID	Out	1	Resposta válida
S_AXI_BREADY	In	1	Aceite da resposta
S_AXI_ARADDR	In	32	Endereço de leitura
S_AXI_ARVALID	In	1	Validação da leitura
S_AXI_ARREADY	Out	1	Pronto para leitura
S_AXI_RDATA	Out	32	Dados de leitura
S_AXI_RRESP	Out	2	Resposta de leitura (fixo OKAY)
S_AXI_RVALID	Out	1	Dados válidos
S_AXI_RREADY	In	1	Aceite da leitura
AXI4 Master Read			
M_AXI_ARADDR	Out	32	Endereço de leitura
M_AXI_ARLEN	Out	8	Comprimento do burst
M_AXI_ARSIZE	Out	3	Tamanho do beat (log2 bytes)
M_AXI_ARBURST	Out	2	Tipo de burst (INCR)
M_AXI_ARVALID	Out	1	Endereço válido
M_AXI_ARREADY	In	1	Aceite do endereço
M_AXI_RDATA	In	32	Dados lidos
M_AXI_RRESP	In	2	Resposta de leitura
M_AXI_RLAST	In	1	Último beat do burst
M_AXI_RVALID	In	1	Dados válidos
M_AXI_RREADY	Out	1	Pronto para receber dados
AXI4 Master Write			
M_AXI_AWADDR	Out	32	Endereço de escrita
M_AXI_AWLEN	Out	8	Comprimento do burst
M_AXI_AWSIZE	Out	3	Tamanho do beat
M_AXI_AWBURST	Out	2	Tipo de burst (INCR)
M_AXI_AWVALID	Out	1	Endereço válido
M_AXI_AWREADY	In	1	Aceite do endereço
M_AXI_WDATA	Out	32	Dados a escrever
M_AXI_WSTRB	Out	4	Máscara (sempre ativa)
M_AXI_WLAST	Out	1	Último beat
M_AXI_WVALID	Out	1	Dados válidos
M_AXI_WREADY	In	1	Aceite dos dados
M_AXI_BRESP	In	2	Resposta de escrita
M_AXI_BVALID	In	1	Resposta válida
M_AXI_BREADY	Out	1	Aceite da resposta
Sinal de Interrupção			
IRQ	Out	1	Interrupção por DONE ou ERROR

No RTL, a interface AXI4-Lite possui largura de dados explicitamente fixada em 32 bits por meio do parâmetro `AXIL_DATA_WIDTH = 32`. Já as larguras de endereço e dados do datapath principal são parametrizadas por `ADDR_WIDTH` e `DATA_WIDTH`. Na configuração efetivamente validada no fluxo de síntese e GLS, o bloco operou com 32 bits de endereço e 32 bits de dados.

1.4 Conformidade com AXI4 e AXI-Lite

Embora o projeto utilize a estrutura básica dos protocolos AXI4 e AXI4-Lite, a implementação é intencionalmente simplificada e não cobre toda a especificação AMBA AXI.

Nos canais AXI4 de leitura e escrita, o controlador suporta apenas bursts incrementais (INCR). Os sinais ARBURST e AWBURST são fixados nesse modo, e o tamanho do beat é derivado diretamente da largura do barramento. Não há suporte a bursts do tipo FIXED ou WRAP. Além disso, o projeto opera com apenas uma transação pendente por vez, com execução sequencial controlada por FSMs, sem sobreposição de bursts.

A implementação não inclui sinais avançados do protocolo, como identificadores de transação (ID), atributos de proteção, cache, QoS ou sinais de usuário, e também não suporta execução fora de ordem ou múltiplas transações outstanding. O fluxo adotado é totalmente ordenado e bloqueante.

As transferências assumem alinhamento com a largura do barramento. O sinal WSTRB é fixo, caracterizando sempre escrita de palavra completa. Dessa forma, não há suporte a transferências estreitas nem a acessos desalinhados.

O tratamento de erro é simplificado: qualquer resposta diferente de OKAY nos sinais RRESP ou BRESP gera uma condição de erro interna. No entanto, não há diferenciação entre tipos de erro, nem mecanismos de recuperação, sendo apenas sinalizada a falha ao software.

Na interface AXI4-Lite, a implementação segue o modelo básico do protocolo, permitindo independência entre os canais de endereço e dados de escrita. Entretanto, as respostas BRESP e RRESP são fixadas em OKAY, mesmo em acessos inválidos, que retornam dados nulos sem sinalização de erro.

O bloco também inclui um sinal de interrupção (IRQ) baseado em eventos de conclusão e erro, configurável via registradores. Embora não faça parte do protocolo AXI, esse mecanismo facilita a integração com sistemas baseados em processador.

Em síntese, o projeto adota a estrutura canônica do AXI, porém com simplificações relevantes: apenas bursts incrementais, ausência de múltiplas transações, suporte limitado a erros e ausência de funcionalidades avançadas. Essas escolhas são coerentes com o escopo didático e a complexidade controlada do projeto.

1.5 Mapa de Registradores

O bloco de controle `dma_csr` implementa um conjunto de registradores acessíveis via AXI4-Lite para configuração do DMA, leitura de status e gerenciamento de interrupção. A escrita no registrador de controle pode gerar pulsos internos como `start_pulse`, `abort_pulse` e `soft_rst_pulse`, enquanto registradores dedicados armazenam os endereços de origem e destino, o tamanho da transferência em bytes e o número máximo de beats por burst.

Do ponto de vista funcional, o mapa de registradores contempla pelo menos os seguintes grupos:

Tabela 2: Mapa de registradores do controlador DMA.

Endereço	Registrador	Acesso	Bits	Descrição
0x00	CTRL	RW	[0]	START – Inicia a transferência
0x00	CTRL	RW	[1]	ABORT – Aborta a operação
0x00	CTRL	RW	[2]	SOFT_RST – Reset lógico do DMA
0x00	CTRL	RW	[8]	IRQ_EN_DONE – Enable IRQ de conclusão
0x00	CTRL	RW	[9]	IRQ_EN_ERR – Enable IRQ de erro
0x04	STATUS	RO/W1C	[0]	BUSY – DMA em execução
0x04	STATUS	RO/W1C	[1]	DONE – Transferência concluída
0x04	STATUS	RO/W1C	[2]	ERR – Erro detectado
0x08	SRC_ADDR	RW	[31:0]	Endereço de origem
0x0C	DST_ADDR	RW	[31:0]	Endereço de destino
0x10	LEN	RW	[31:0]	Tamanho da transferência em bytes
0x14	BURST	RW	[7:0]	Número máximo de beats por burst
0x18	ERROR	RO	[7:0]	Código de erro latched
0x1C	IRQ_STATUS	RW1C	[0]	DONE_IRQ – Flag de conclusão
0x1C	IRQ_STATUS	RW1C	[1]	ERR_IRQ – Flag de erro

A leitura dos registradores é feita diretamente a partir do endereço AXI-Lite. A escrita utiliza aplicação seletiva de WSTRB, permitindo atualização por byte nos registradores

de configuração. Já os registradores de status e interrupção incluem semânticas de limpeza por escrita, usadas para reconhecimento de eventos e retorno da FSM ao estado ocioso após conclusão ou erro.

2 Implementação RTL

2.1 Estratégia de Design

A implementação do controlador DMA foi baseada em uma arquitetura modular, com separação clara entre controle, datapath e interface com o protocolo AXI. O objetivo principal foi manter o design simples, determinístico e facilmente verificável, ao mesmo tempo em que preserva a estrutura fundamental de um DMA real.

O bloco foi particionado em três componentes principais: uma máquina de estados central responsável pelo controle da operação, dois módulos AXI independentes para leitura e escrita, e uma FIFO intermediária para desacoplamento entre os canais. Essa abordagem permite separar responsabilidades e simplificar tanto a lógica quanto a verificação.

A execução do DMA segue um modelo sequencial, no qual apenas um burst é processado por vez. A FSM principal calcula dinamicamente o tamanho do burst com base no comprimento restante da transferência e no limite configurado, coordenando a ativação dos módulos de leitura e escrita. Essa escolha evita a complexidade associada a múltiplas transações simultâneas e elimina a necessidade de controle de reorder ou múltiplos IDs.

O uso de uma FIFO interna permite desacoplar os canais de leitura e escrita, evitando dependência direta entre os handshakes AXI. Isso melhora a robustez frente a back-pressure e simplifica o controle da FSM, já que os motores podem operar de forma parcialmente independente.

2.2 Hierarquia do Projeto

A hierarquia do projeto é composta pelos seguintes módulos principais:

- **dma_axi_top**: módulo de topo, responsável pela integração de todos os blocos e pela conexão com as interfaces AXI externas;
- **dma_fsm**: máquina de estados principal, responsável pelo controle da transferência e coordenação dos motores;

- **dma_axi_master_rd**: motor de leitura AXI, responsável por emitir transações AR e consumir dados do canal R;
- **dma_axi_master_wr**: motor de escrita AXI, responsável por emitir transações AW/W e tratar respostas no canal B;
- **dma_fifo**: buffer intermediário para armazenamento temporário dos dados entre leitura e escrita;
- **dma_csr**: bloco de registradores de configuração, status e controle via AXI-Lite.

O módulo de topo conecta esses blocos por meio de sinais internos de comando e status, incluindo:

- sinais de início de operação (`start_pulse`);
- parâmetros de transferência (endereços, comprimento e burst);
- controle de fluxo da FIFO (`valid/ready`);
- flags de status (`busy`, `done`, `error`).

2.3 Máquina de Estados (FSM)

O controle da transferência é realizado por uma FSM central implementada no módulo `dma_fsm`. Essa FSM opera de forma sequencial, coordenando a emissão de comandos de leitura e escrita até que todo o comprimento configurado seja transferido.

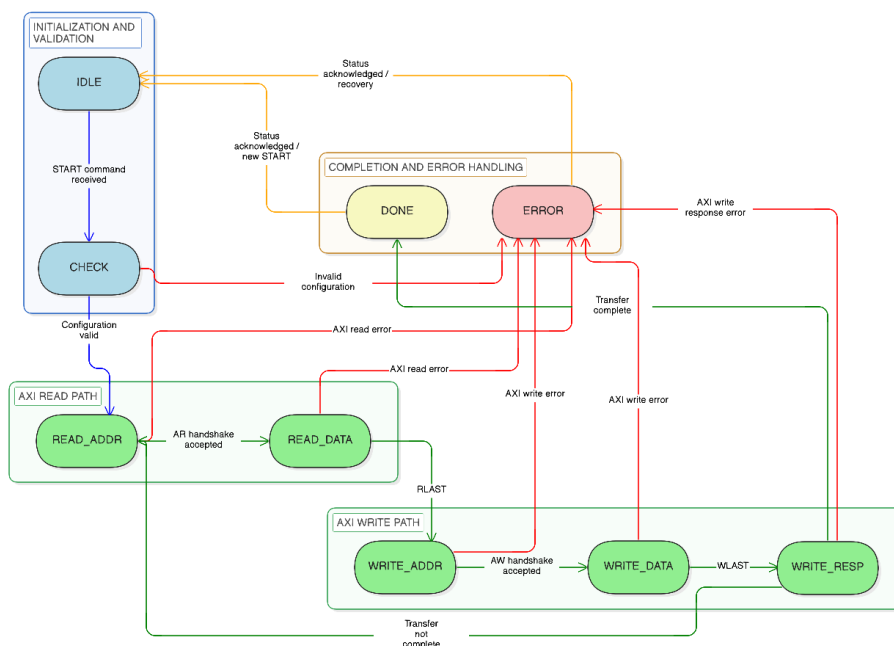


Figura 2: Máquina de estados do controlador DMA.

O comportamento da FSM pode ser descrito pelos seguintes estados principais:

- **IDLE**: aguarda comando de início (`start_pulse`);
- **READ_REQ**: emite comando de leitura para o motor AXI;
- **READ_DATA**: aguarda recepção dos dados e preenchimento da FIFO;
- **WRITE_REQ**: emite comando de escrita baseado nos dados disponíveis na FIFO;
- **WRITE_DATA**: realiza envio dos dados para o canal AXI de escrita;
- **DONE**: sinaliza conclusão da transferência;
- **ERROR**: sinaliza ocorrência de erro e interrompe a operação.

A FSM mantém contadores internos de bytes transferidos e controla dinamicamente o tamanho dos bursts. O término da operação ocorre quando o comprimento total configurado é atingido ou quando uma condição de erro é detectada.

2.4 Motores AXI

Os motores de leitura e escrita foram implementados como FSMs independentes, cada uma responsável por um subconjunto dos canais AXI.

O motor de leitura segue a sequência:

IDLE → AR → RDATA

Nesse fluxo, o endereço é enviado no canal AR e os dados são recebidos no canal R até a chegada de RLAST. O controle de RREADY é mantido ativo enquanto a FIFO possui espaço disponível.

O motor de escrita segue a sequência:

IDLE → AW → WDATA → BRESP

Primeiramente o endereço é enviado no canal AW, seguido pela transmissão dos dados no canal W. O sinal WLAST é utilizado para indicar o último beat do burst. Após o envio completo, a FSM aguarda a resposta no canal B.

Ambos os motores operam com apenas um burst ativo por vez, sem pipeline de comandos ou suporte a múltiplas transações concorrentes.

2.5 Pipeline e Desacoplamento de Dados

A arquitetura do controlador DMA pode ser interpretada como um pipeline simples de dois estágios principais: leitura e escrita. Esses estágios são desacoplados por meio

de uma FIFO intermediária, permitindo operação parcialmente independente entre os motores AXI.

O estágio de leitura atua como produtor de dados, realizando transações no canal AXI de leitura e alimentando a FIFO. Já o estágio de escrita atua como consumidor, retirando dados da FIFO e realizando transações no canal AXI de escrita.

Esse desacoplamento introduz um comportamento de pipeline no sistema, no qual a leitura de novos dados pode ocorrer enquanto dados previamente recebidos ainda estão sendo transmitidos para o destino. Isso melhora a utilização das interfaces AXI e reduz dependências diretas entre os canais.

A FIFO atua como um *elastic buffer*, absorvendo variações de latência e backpressure entre os canais de leitura e escrita. Caso o canal de escrita esteja momentaneamente indisponível, os dados podem ser armazenados temporariamente, evitando a necessidade de interromper imediatamente o fluxo de leitura.

Na implementação atual, a FIFO opera sob um único domínio de clock (ACLK), caracterizando uma FIFO síncrona. Portanto, não há necessidade de mecanismos de sincronização entre domínios de clock (*Clock Domain Crossing*).

Essa abordagem mantém o design simples e eficiente, ao mesmo tempo em que preserva as vantagens de desacoplamento e tolerância a variações de throughput entre os estágios do pipeline.

2.6 Considerações de Projeto para ASIC

A implementação foi realizada seguindo boas práticas de projeto sintetizável:

- todo o design é síncrono a um único clock (ACLK);
- reset ativo em nível baixo (ARESETn);
- uso exclusivo de lógica sequencial e combinacional sintetizável;
- ausência de *latches* inferidos;
- FSMs codificadas de forma explícita e determinística;
- separação clara entre controle e datapath;
- uso de parâmetros para configuração de largura de dados e profundidade da FIFO.

Além disso, o design evita construções não suportadas por ferramentas de síntese e mantém uma estrutura compatível tanto com fluxos open-source (Yosys) quanto com ferramentas comerciais (Genus).

2.7 Trechos Relevantes de Código

A seguir, o trecho da seleção de estados da FSM:

```
// Core do FSM
```

```
always_comb begin
```

```
    state_n      = state;
```

```
    rd_cmd_valid = 1'b0;
```

```
    wr_cmd_valid = 1'b0;
```

```
case (state)
```

```
    ST_IDLE:
```

```
        if (start_pulse) state_n = ST_CHECK;
```

```
    ST_CHECK:
```

```
        if (abort_pulse || !cfg_ok) state_n = ST_ERROR;
```

```
        else state_n = ST_READ_ADDR;
```

```
    ST_READ_ADDR: begin
```

```
        rd_cmd_valid = 1'b1;
```

```
        if (abort_pulse) state_n = ST_ERROR;
```

```
        else if (rd_cmd_ready) state_n = ST_READ_DATA;
```

```
    end
```

```
    ST_READ_DATA: begin
```

```
        if (abort_pulse)
```

```
            state_n = ST_ERROR;
```

```
        else if (rd_done) begin
```

```
            if (rd_err)
```

```
                state_n = ST_ERROR;
```

```
            else if (wr_addr_issued_q)
```

```
                state_n = ST_WRITE_RESP;
```

```
            else
```

```
                state_n = ST_WRITE_ADDR;
```

```
        end
```

```
        else if (pipeline_wr_ready && !wr_addr_issued_q && !wr_busy)
```

```
            state_n = ST_PIPE_WR_ADDR;
```

```
    end
```

```
    ST_PIPE_WR_ADDR: begin
```

```
        wr_cmd_valid = 1'b1;
```

```

    if (abort_pulse)
        state_n = ST_ERROR;
    else if (wr_cmd_ready)
        state_n = ST_READ_DATA;
end

ST_WRITE_ADDR: begin
    wr_cmd_valid = 1'b1;
    if (abort_pulse) state_n = ST_ERROR;
    else if (wr_cmd_ready) state_n = ST_WRITE_RESP;
end

ST_WRITE_RESP: begin
    if (abort_pulse)
        state_n = ST_ERROR;
    else if (wr_done) begin
        if (wr_err)
            state_n = ST_ERROR;
        else if (rem_next == '0)
            state_n = ST_DONE;
        else
            state_n = ST_READ_ADDR;
        end
    end
end

ST_DONE: if (status_ack) state_n = ST_IDLE;
ST_ERROR: if (status_ack) state_n = ST_IDLE;
default: state_n = ST_IDLE;

endcase
end

```

3 Estratégia de Verificação e Resultados

3.1 Ambiente UVM

A verificação do controlador DMA foi realizada utilizando a metodologia UVM (Universal Verification Methodology), com um ambiente modular estruturado em agentes, sequenciadores, monitores, modelo de referência e scoreboard.

O ambiente desenvolvido inclui um agente AXI-Lite responsável pela configuração do

DUT, composto por driver, monitor e sequencer. As transações de configuração são geradas por sequências UVM e enviadas ao DUT através desse agente.

O ambiente também incorpora um modelo de memória (`axi_mem_model`), utilizado como referência funcional para armazenar os dados transferidos e permitir a verificação do comportamento do DMA.

A coordenação das sequências é realizada por um *virtual sequencer*, permitindo a execução de cenários mais complexos e a combinação de diferentes estímulos.

3.2 Golden Model e Scoreboard

A verificação funcional do DMA é baseada em um modelo de referência implícito, implementado por meio da memória AXI e do scoreboard.

O scoreboard compara diretamente os dados armazenados nas regiões de memória de origem e destino após a execução da transferência. Para cada operação de DMA, é verificado se os dados foram corretamente copiados, respeitando endereço, comprimento e organização em bursts.

Os logs de simulação indicam que, para todos os cenários testados, a verificação foi bem-sucedida, conforme evidenciado por mensagens como:

```
[SB] DMA copy check passed: src=00120000 dst=00130000 len=256
```

Além disso, o scoreboard também publica informações de cobertura funcional, incluindo parâmetros como endereços, tamanho da transferência, número de bursts e status final da operação.

3.3 Verificação Baseada em Assertions (SVA)

O ambiente inclui um conjunto de assertions SystemVerilog para monitorar propriedades críticas do protocolo e do controle do DMA, incluindo:

- verificação da interface AXI-Lite;
- consistência do controle interno do DMA;
- comportamento dos canais AXI de leitura;
- comportamento dos canais AXI de escrita.

As assertions são conectadas ao DUT por meio de mecanismos de *bind*, permitindo observabilidade sem modificar o RTL original.

Durante a regressão, um total de 58 assertions foram instrumentadas. Os resultados indicam ausência de falhas em todos os seeds executados:

- Total de assertions (por seed): 58
- Falhas: 0
- Assertions não exercitadas: 22

Esses resultados indicam que as propriedades verificadas foram respeitadas pelo design nos cenários exercitados.

3.4 Análise de Cobertura

3.4.1 Code Coverage

A instrumentação de cobertura foi configurada para coletar métricas de bloco, expressão, FSM, toggle e cobertura funcional. Durante a elaboração, foram identificadas FSMs nos principais módulos do design, incluindo:

- dma_fsm
- dma_axi_master_rd
- dma_axi_master_wr

Essas FSMs foram automaticamente extraídas para análise de cobertura, permitindo verificar a ativação dos estados durante a simulação.

3.4.2 Functional Coverage

A cobertura funcional foi implementada por meio de um coletor dedicado (dma_cov_collector), responsável por amostrar cenários relevantes de operação, incluindo:

- transferências curtas e longas;
- diferentes tamanhos de burst;
- número de bursts por operação;
- casos de último burst parcial;
- operações com e sem erro.

Os logs indicam a coleta de múltiplos cenários distintos, como:

```
[COV] sampled coverage item: src=00140000 dst=00150000 len=256  
↪ max_beats=16 num_bursts=4
```

Além disso, diferentes padrões de transferência foram exercitados ao longo dos seeds, incluindo casos mínimos, máximos e intermediários, indicando boa diversidade de estímulos.

3.5 Resultados de Simulação RTL

A validação funcional foi realizada por meio de testes UVM direcionados e pseudoaleatórios, incluindo:

- testes básicos de funcionalidade;
- testes com variação de tamanho de burst;
- testes com transferências longas;
- testes com bursts parciais;
- *corner cases*;

A regressão executada com o teste `dma_corner_cases_test` foi realizada com 5 seeds independentes, apresentando taxa de sucesso de 100%:

- Seeds executados: 5
- Testes aprovados: 5
- Falhas: 0

Além disso, nenhuma assertion falhou durante a regressão, indicando consistência do comportamento observado.

A Figura 3 ilustra uma transferência típica do controlador DMA, destacando a interação entre os canais AXI de leitura e escrita. Observa-se a emissão de bursts incrementais no canal de leitura (AR/R), seguida pelo armazenamento temporário dos dados na FIFO e posterior envio no canal de escrita (AW/W/B).

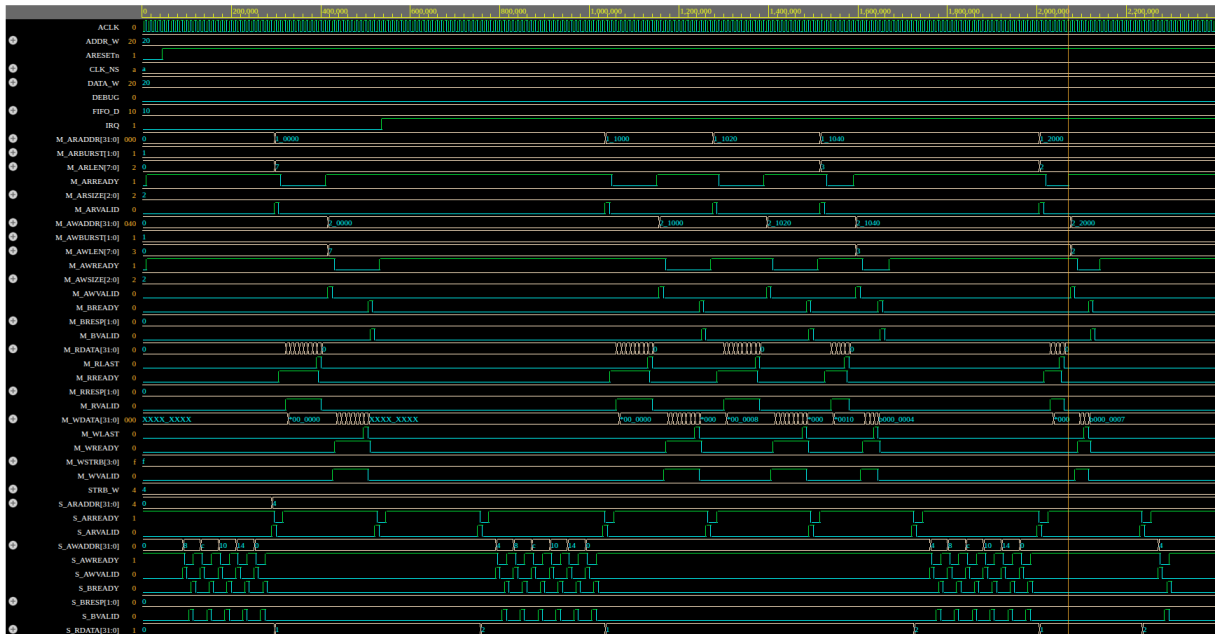


Figura 3: Waveform representativa de uma transferência DMA em burst, evidenciando a coordenação entre os canais AXI de leitura e escrita.

É possível verificar a correta sincronização dos sinais de handshake (VALID/READY), bem como a indicação de término de burst através de RLAST e WLAST. A sobreposição parcial entre leitura e escrita evidencia o desacoplamento proporcionado pela FIFO, caracterizando o comportamento de pipeline do sistema.

Além disso, nota-se a consistência entre os dados lidos (RDATA) e escritos (WDATA), confirmando a funcionalidade de cópia de memória implementada pelo controlador.

Os resultados obtidos demonstram que o design apresenta comportamento funcional correto no nível RTL, cobrindo uma variedade relevante de cenários operacionais.

4 Síntese Lógica e Resultados PPA

4.1 Fluxo de Síntese

A síntese lógica do controlador DMA foi realizada utilizando a ferramenta open-source Yosys, com mapeamento tecnológico para a biblioteca padrão Sky130 (sky130_fd_sc_hd). O fluxo incluiu as etapas de leitura do RTL, elaboração, otimização, mapeamento tecnológico e geração da netlist final.

Durante a síntese, todos os módulos principais do design foram corretamente instanciados, incluindo os motores AXI, a FSM central, o bloco de registradores e a FIFO interna. O processo de elaboração confirmou a correta hierarquia do projeto e a parametrização

dos módulos, com largura de dados de 32 bits e profundidade de FIFO configurada para 16 entradas.

4.2 Restrições de Projeto

As restrições de temporização foram definidas por meio de um arquivo SDC, especificando um único domínio de clock (ACLK) com período de 10 ns, correspondente a uma frequência alvo de 100 MHz.

O design utiliza reset assíncrono ativo em nível baixo, aplicado a todos os registradores principais. Não foram especificadas restrições adicionais de I/O ou múltiplos domínios de clock, refletindo a simplicidade arquitetural do sistema.

4.3 Validação Pós-Síntese por Gate-Level Simulation

A validação pós-síntese foi realizada por meio de simulação *gate-level* (GLS), utilizando duas abordagens distintas: uma com netlist genérica (sem biblioteca tecnológica) e outra com netlist mapeada para a biblioteca padrão Sky130.

Na simulação com netlist genérica, o design apresentou comportamento funcional consistente com o RTL. Os canais AXI-Lite operaram corretamente, com handshakes completos de escrita e leitura, e a configuração do DMA foi realizada com sucesso. Durante a execução, observou-se a emissão de bursts no canal de leitura e a evolução do status do controlador, indicando operação ativa (*busy*=1).

Por outro lado, na simulação com a biblioteca Sky130, o design apresentou falha logo nas primeiras transações de configuração via AXI-Lite. Após a sequência de reset, os sinais de handshake do barramento (*AWREADY*, *WREADY*, *ARREADY*) permaneceram indefinidos (x), impedindo o avanço da transação. Como consequência, o testbench reportou timeout na escrita de registradores:

```
[TB][FAIL] csr_write timeout AW/W AWREADY=x WREADY=x
```

Esse comportamento indica que o DUT não respondeu corretamente aos estímulos após a inicialização, bloqueando o progresso da simulação.

Adicionalmente, a simulação com a biblioteca tecnológica apresentou múltiplos avisos relacionados a temporização e sinais internos, incluindo ausência de suporte a *timing checks* e sinais derivados não dirigidos.

4.3.1 Análise do Problema

A discrepância entre os resultados de RTL e GLS indica uma inconsistência entre o modelo comportamental e a implementação sintetizada. Um dos principais indícios é a propagação de valores indefinidos (x) nos sinais de controle do barramento logo após o reset.

Esse comportamento é tipicamente associado a:

- inicialização incompleta de registradores na lógica sintetizada;
- dependência de valores implícitos assumidos no RTL (como inicialização em zero);
- diferenças na modelagem de reset entre RTL e netlist;
- propagação de estados desconhecidos em lógica combinacional.

No caso observado, o fato de os sinais READY permanecerem indefinidos sugere que a FSM ou os registradores de controle responsáveis por esses sinais não estão sendo corretamente inicializados após o reset na implementação sintetizada.

Além disso, a ausência de suporte completo a *timing checks* no simulador e o uso de modelos comportamentais simplificados da biblioteca podem contribuir para a propagação de valores indefinidos, agravando o problema.

4.4 Resultados de Timing

A análise estática de temporização foi realizada com OpenSTA sobre a netlist mapeada para a biblioteca sky130_fd_sc_hd, considerando um clock de 10 ns (100 MHz). Os resultados indicam atendimento à restrição temporal, com:

- Worst Negative Slack (WNS): 0.00 ns
- Total Negative Slack (TNS): 0.00 ns

Embora os relatórios resumidos apresentem WNS e TNS nulos, a inspeção detalhada do caminho crítico mostra um atraso de chegada de dados de aproximadamente 8.49 ns para um tempo requerido de 9.67 ns, resultando em slack positivo de cerca de 1.17 ns. Isso confirma que o design atende às restrições de timing impostas.

O caminho crítico identificado está inteiramente concentrado no módulo u_fsm, entre dois flip-flops acionados por ACLK. A cadeia crítica é composta majoritariamente por lógica combinacional de controle, incluindo portas como maj3, or4, nor4, mux2 e lógica AOI/OAI, indicando que o principal limitante de frequência do sistema é a lógica de decisão da máquina de estados, e não o datapath de dados propriamente dito.

Esse resultado é coerente com a arquitetura adotada, na qual a FSM central coordena emissão de bursts, controle de FIFO, avanço de endereços e sinalização de término. Em versões futuras, uma possível estratégia de otimização seria reduzir a profundidade lógica do caminho de controle ou particionar parte da lógica da FSM para melhorar a frequência máxima de operação.

4.5 Discussão de Área

Durante a síntese, foi observado que a FIFO interna foi mapeada como um conjunto de registradores (*flip-flops*), em vez de memória dedicada. Esse comportamento é evidenciado diretamente no log do sintetizador:

Warning: Replacing memory \mem with list of registers

A análise de área da implementação mapeada para a biblioteca Sky130 indica que o módulo de topo `dma_axi_top` apresenta área total de 40352.45 unidades de área da biblioteca. A distribuição por submódulo é dada por:

- `dma_fifo`: 20692.35
- `dma_fsm`: 7937.61
- `dma_csr`: 7781.21
- `dma_axi_master_wr`: 2305.96
- `dma_axi_master_rd`: 1635.32

Esses resultados mostram que a FIFO é o bloco dominante em área, representando aproximadamente 51% da área total do circuito.

Do ponto de vista de contagem de células, a FIFO armazena 512 bits ($32 \text{ bits} \times 16$ palavras), o que implica diretamente na utilização de pelo menos 512 flip-flops apenas para armazenamento de dados. Considerando também registradores de controle (FSM, CSR e lógica AXI), o número total de elementos sequenciais é da ordem de algumas centenas adicionais, resultando em aproximadamente 800 a 1000 flip-flops no design.

A lógica combinacional é composta por algumas centenas de portas padrão. A análise do mapeamento tecnológico revela, por exemplo:

- 122 instâncias de `nand2`;
- 19 instâncias de `nor2`;
- 14 instâncias de `a21oi`;

-
- múltiplos multiplexadores e células AOI/OAI;

indicando um datapath relativamente leve e uma lógica de controle concentrada na FSM.

Na biblioteca Sky130 HD, células sequenciais típicas como `sky130_fd_sc_hd__df r tp_1` apresentam área da ordem de 25 unidades, o que explica o impacto significativo da FIFO no resultado final.

Assim, o design pode ser caracterizado como dominado por elementos sequenciais, com a FIFO sendo o principal contribuinte de área, enquanto a lógica combinacional apresenta complexidade moderada.

Em implementações mais avançadas, especialmente para maiores profundidades de buffer, seria desejável utilizar memória dedicada (*memory macros*), reduzindo significativamente tanto a área quanto o número total de células.

4.6 Discussão de Potência

A análise de potência não foi realizada neste trabalho. No entanto, com base na estrutura do design, espera-se que o consumo seja dominado pela atividade dos registradores da FIFO e pela lógica combinacional associada à FSM.

A ausência de clock gating e o uso de registradores em vez de memória dedicada contribuem para um consumo potencialmente maior em comparação a implementações otimizadas para baixo consumo.

5 Conclusão e Lições Aprendidas

Este trabalho apresentou o desenvolvimento completo de um controlador DMA do tipo *Memory-to-Memory* baseado em protocolo AXI, cobrindo todas as etapas do fluxo de front-end, desde a especificação e implementação RTL até verificação funcional, síntese lógica e simulação gate-level.

Os resultados obtidos demonstram que o design atende corretamente aos requisitos funcionais em nível RTL, com validação robusta por meio de um ambiente UVM estruturado, incluindo testes direcionados, pseudoaleatórios, assertions e cobertura funcional. A consistência dos resultados reforça a corretude do modelo comportamental.

No fluxo de síntese, o projeto apresentou conformidade com as restrições de temporização, atingindo a frequência alvo com slack positivo. A análise do caminho crítico evidenciou a FSM como principal limitante de desempenho, enquanto a implementação da FIFO em registradores mostrou impacto significativo na área total do circuito.

A simulação gate-level revelou divergências em relação ao comportamento observado no RTL, especialmente relacionadas à inicialização e propagação de valores indefinidos. Esses resultados evidenciam diferenças fundamentais entre modelos comportamentais e estruturais, destacando a importância da validação pós-síntese para identificação de problemas não visíveis em nível RTL.

Como trabalhos futuros, destacam-se a introdução de pipeline mais profundo, suporte a múltiplas transações concorrentes e utilização de memória dedicada para a FIFO, visando melhorias em desempenho, área e escalabilidade.

De forma geral, o projeto proporcionou uma visão integrada do desenvolvimento de sistemas digitais, consolidando conhecimentos em arquitetura, protocolos AXI, verificação baseada em UVM e fluxo de síntese para ASICs.

Referências

- [1] V. S. Oliveira, *AXI DMA Controller Repository*, GitHub, 2026. Disponível em: <https://github.com/vinicius-soliveira/axi-dma>
- [2] EDA Playground, *Directed Testbench for AXI DMA*, 2026. Disponível em: <https://www.edaplayground.com/x/eBju>
- [3] EDA Playground, *UVM Testbench for AXI DMA*, 2026. Disponível em: <https://www.edaplayground.com/x/GgdV>
- [4] ARM Ltd., *AMBA Specifications Overview*. Disponível em: <https://www.arm.com/architecture/system-architectures/amba/amba-specifications>
- [5] ARM Ltd., *AMBA AXI and ACE Protocol Specification*. Disponível em: <https://documentation-service.arm.com/static/63ff0ebd56ea36189d4e7ee7>
- [6] Advanced eXtensible Interface (AXI) Overview. Disponível em: https://en.wikipedia.org/wiki/Advanced_eXtensible_Interface
- [7] Overview of AMBA AXI Protocol Features. Disponível em: <https://www.scribd.com/document/580678390/AMBAaxi-20>
- [8] Introduction to AXI4-Lite Interface. Disponível em: <https://www.realdigital.org/doc/a9fee931f7a172423e1ba73f66ca4081>
- [9] Yosys Open SYnthesis Suite. Disponível em: <https://yosyshq.net/yosys/>
- [10] OpenSTA Static Timing Analyzer. Disponível em: <https://github.com/The-OpenROAD-Project/OpenSTA>
- [11] SkyWater 130nm PDK Documentation. Disponível em: <https://skywater-pdk.readthedocs.io/>
- [12] Accellera Systems Initiative, *Universal Verification Methodology (UVM)*, 2020. Disponível em: <https://www.accellera.org/downloads/standards/uvm>